

A CLEAN, SIMPLE LEARNING GUIDE

FastAPI Handbook

From first "Hello" to secure, async, production-ready APIs

Instagram: @____raviiiiiii

What is this PDF?

This handbook is a clean, step-by-step summary of the core ideas of **FastAPI**, a modern, high-performance Python framework for building APIs. Every chapter is one page: a short explanation, simple code, and key takeaways, so you can learn one topic at a time and revise quickly.

Who is it for?

- Beginners who know basic Python and want to start building APIs.
- Students and developers who want a quick, organised revision guide.
- Anyone moving from Flask or Django who wants modern, typed, async-ready APIs.

What is covered?

The guide moves in a natural order: understand FastAPI, set it up, structure the project, then build endpoints with parameters, request bodies and responses, validate data, handle headers, cookies and forms, secure the API, add middleware and CORS, handle errors, and finally use async programming and background tasks.

Table of contents

Page	Chapter	What you learn
2	1. FastAPI Introduction + Why FastAPI	What FastAPI is, tech stack, comparison with Flask and Django, use cases
3	2. Installation + Project Setup	Prerequisites, install, virtual environment, first main.py, run the server
4	3. Project Structure + Application Instance	Recommended folders, FastAPI() settings, auto docs, next steps
5	4. Path Operations + HTTP Methods	GET, POST, PUT, PATCH, DELETE and the request-response flow
6	5. Path Parameters + Query Parameters	Dynamic URLs, filters, optional and required parameters
7	6. Request Body + Pydantic Models	Receive JSON, validation rules, nested models, lists
8	7. Response Models + Status Codes	Shape responses, status codes, JSONResponse
9	8. Pydantic Validation + Schemas	Field(), conint, EmailStr, separate request and response schemas
10	9. Headers, Cookies + Form Data	Header(), Cookie(), Form(), file uploads
11	10. Authentication + OAuth2 + JWT	Password hashing, JWT tokens, login, protected routes
12	11. Middleware + CORS	Custom middleware, class-based middleware, CORS for frontends
13	12. Exception Handling + Custom Errors	HTTPException, custom exceptions, global handlers
14	13. Async Programming + Background Tasks	async/await, when to use async, BackgroundTasks, limits
15	Quick Revision Cheat Sheet	Everything important on one page

How to use this guide: read a chapter, type the code yourself, run it with uvicorn, and test it in the Swagger UI at /docs. Page 15 is a one-page revision sheet.

FastAPI Introduction + Why FastAPI

"Less code. More features. Higher performance."

What is FastAPI?

- A modern, high-performance Python web framework for building APIs.
- Built on top of **Starlette** (web handling) and **Pydantic** (data validation).
- Lets you build robust, production-ready APIs with less code and more features.
- Supports type hints, automatic documentation and async programming.
- Designed to be fast, easy to learn and developer friendly.

Why FastAPI?

- High performance, comparable to Node.js and Go. Easy to learn and use.
- Automatic interactive API documentation. Built-in data validation (Pydantic).
- Supports async and await. Uses type hints and modern Python features.
- Modular and flexible. Production ready and used by big companies. Excellent community support.

Technology stack

Tool	Role
FastAPI	Main framework (built on Starlette)
Starlette	Handles the web layer: routing, requests, responses
Pydantic	Data validation using Python type hints
Uvicorn	ASGI server that runs the application

FastAPI vs traditional frameworks

Feature	FastAPI	Flask	Django
Performance	Very high	Moderate	Moderate
Data validation	Automatic (Pydantic)	Manual	Manual
Documentation	Auto generated	Needs extension	Needs setup
Type hints	Built in	Not by default	Partial
Async support	Yes	Limited	Partial
Learning curve	Easy	Easy	Steeper
Use case	APIs	Small / medium projects	Large projects

Simple example

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def read_root():
    return {"message": "Hello from FastAPI!"}
```

The app object is created, a route / is defined with the `@app.get` decorator, and visiting `http://127.0.0.1:8000/` returns a JSON response.

Real world use cases

REST APIs and microservices • Machine learning model deployment • Real-time apps (WebSockets) • Internal tools and dashboards • Mobile and frontend backends • Serverless functions

Key takeaways

- ✓ FastAPI is a modern and fast Python framework, built on Starlette and Pydantic.
- ✓ It provides automatic documentation and data validation, and is easy to learn.

Installation + Project Setup

"Set up once. Build anything with FastAPI."

1. Prerequisites

- Basic knowledge of Python; Python 3.8 or later (3.10+ recommended).
- pip (Python package manager), a code editor (VS Code recommended), a terminal / command prompt, Git (optional).

2. Install FastAPI and Uvicorn

```
pip install fastapi          # installs FastAPI
pip install "uvicorn[standard]" # installs Uvicorn (ASGI server) with standard dependencies
```

3. Verify installation

```
python --version    # e.g. Python 3.10.12
pip --version       # e.g. pip 23.2.1
fastapi --version   # e.g. fastapi 0.110.0
uvicorn --version   # e.g. uvicorn 0.29.0
```

4. Create a new project

```
mkdir fastapi-app    # create project folder
cd fastapi-app       # move into the project folder
```

5. Create a virtual environment (recommended)

```
python -m venv .venv          # create virtual environment
.venv\Scripts\activate        # activate (Windows)
source .venv/bin/activate      # activate (macOS / Linux)
```

After activation you will see **(.venv)** in your terminal, which means the environment is active.

6. Install dependencies again, inside the venv

```
pip install fastapi
pip install "uvicorn[standard]"
```

7. Create the project structure

```
fastapi-app/
|-- app/
|   |-- main.py          # main application file
|   |-- __init__.py      # makes app a Python package
|-- .venv/               # virtual environment folder
|-- requirements.txt      # list of dependencies (optional)
|-- README.md            # project documentation (optional)
```

8. Create main.py

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def root():
    return {"message": "Hello from FastAPI!"}
```

9. Run the development server

```
uvicorn app.main:app --reload
```

Open `http://127.0.0.1:8000` in your browser. You should see: `{"message": "Hello from FastAPI!"}`

Small setup, big possibilities.

Project Structure + Application Instance

"A clean structure today saves hours tomorrow."

1. Recommended project structure

```
fastapi-app/           # project root folder
|-- app/               # application package
|   |-- __init__.py    # marks it as a Python package
|   |-- main.py        # FastAPI app instance
|   |-- routers/       # API route files
|       |-- __init__.py
|       |-- users.py    # user related routes
|-- models/           # database models (SQLAlchemy)
|   |-- __init__.py
|   |-- user.py        # user model
|-- schemas/          # Pydantic schemas (request / response)
|   |-- __init__.py
|   |-- user.py        # user schema
|-- services/         # business logic / services
|   |-- __init__.py
|   |-- user_service.py # user service functions
|-- core/             # configuration files
|   |-- __init__.py
|   |-- config.py      # app settings (env variables, etc.)
-- .venv/             # virtual environment
-- requirements.txt    # project dependencies
-- README.md          # project documentation
```

Keep it simple in the beginning and add more folders as your project grows.

2. Create the structure (commands)

```
mkdir fastapi-app && cd fastapi-app
python -m venv .venv
.venv\Scripts\activate      # Windows
source .venv/bin/activate   # macOS / Linux

mkdir app app/routers app/models app/schemas app/services app/core
type nul > app/__init__.py   # Windows
touch app/__init__.py       # macOS / Linux
```

3. Application instance (main.py)

```
from fastapi import FastAPI

app = FastAPI(
    title="My FastAPI App",
    description="A simple and scalable FastAPI project",
    version="1.0.0"
)

@app.get("/")
def read_root():
    return {"message": "Hello from FastAPI!"}
```

This is the main application instance. It creates a FastAPI app and defines a simple root route.

4. Run the project

```
uvicorn app.main:app --reload
```

Part	Meaning
app.main	Path to your main.py file
app	The FastAPI application instance
--reload	Auto reload on code changes (useful in development)

After running, open <http://127.0.0.1:8000> in your browser to see the API response.

5. Check API documentation

FastAPI automatically generates interactive docs. **Swagger UI:** <http://127.0.0.1:8000/docs> **ReDoc:** <http://127.0.0.1:8000/redoc>. No extra configuration needed.

6. Next steps

- ✓ Create routers (app/routers/), add models (app/models/), define schemas (app/schemas/).
- ✓ Add services (app/services/), use environment variables (app/core/config.py).
- ✓ Connect a database and build more API endpoints.

Path Operations + HTTP Methods

“Path operations define what happens when a client sends a request to a specific URL.”

1. What are path operations?

- A path operation is a function that handles an HTTP request for a specific path (URL).
- You define them with decorators like `@app.get()` and `@app.post()`.
- Each path operation can have a path, HTTP method, parameters, request body and response.

2. Basic example

```
from fastapi import FastAPI
app = FastAPI()

@app.get("/")
def read_root():
    return {"message": "Hello from FastAPI!"}

@app.get("/about")
def about():
    return {"message": "This is About page"}
```

Each function is a path operation. `@app.get("/")` means a GET request to the root path (/).

3. Common HTTP methods

Method	Purpose	Use case	Idempotent
GET	Read data	Fetch resources	Yes
POST	Create data	Create a new resource	No
PUT	Update data (full)	Replace entire resource	Yes
PATCH	Update data (partial)	Update specific fields	No
DELETE	Delete data	Remove a resource	Yes

4. Example with all HTTP methods

```
@app.get("/items")
def get_items():
    return {"message": "Get all items"}

@app.post("/items")
def create_item():
    return {"message": "Create an item"}

@app.put("/items/1")
def update_item():
    return {"message": "Update item 1"}

@app.patch("/items/1")
def partial_update_item():
    return {"message": "Partially update item 1"}

@app.delete("/items/1")
def delete_item():
    return {"message": "Delete item 1"}
```

The same path can have different HTTP methods.

5. Run and test

```
uvicorn main:app --reload
# open http://127.0.0.1:8000/docs and test endpoints with Swagger UI
```

6. Request and response flow

Client (Browser / Postman)	HTTP Request (GET / POST etc.)	FastAPI (Path Operation)	Response (JSON)	Client receives data
----------------------------	--------------------------------	--------------------------	-----------------	----------------------

7. Key takeaways

- ✓ Path operations handle HTTP requests; use decorators (`@app.get()`, `@app.post()`, etc.).
- ✓ FastAPI supports all standard HTTP methods; each path can have only one function per method.
- ✓ Test your APIs using the interactive docs. Keep your endpoints clean and meaningful.

Path Parameters + Query Parameters

"Dynamic routes make flexible APIs."

1. Path parameters

Part of the URL path, used to identify a specific resource. Defined with curly braces {} and they are **required**.

```
@app.get("/users/{user_id}")
def get_user(user_id: int):
    return {"user_id": user_id, "message": "User details"}
# Example URL: /users/101
```

2. Multiple path parameters

```
@app.get("/users/{user_id}/posts/{post_id}")
def get_post(user_id: int, post_id: int):
    return {"user_id": user_id, "post_id": post_id, "message": "Post details"}
```

3. Path parameter with data types

You can define types like int, str, float, bool, UUID. FastAPI automatically validates the type.

```
@app.get("/products/{product_id}")
def get_product(product_id: int):
    return {"product_id": product_id}

@app.get("/products/{name}")
def get_product_by_name(name: str):
    return {"name": name}
```

4. Query parameters

Extra parameters passed in the URL after a ?. They filter, search or modify the response, are key-value pairs, and are optional unless required.

```
@app.get("/items")
def get_items(skip: int = 0, limit: int = 10):
    return {"skip": skip, "limit": limit, "message": "List of items"}
# Example URL: /items?skip=0&limit=10
```

5. Multiple query parameters

```
@app.get("/search")
def search(q: str, category: str = None, limit: int = 10):
    return {"query": q, "category": category, "limit": limit}
# Example URL: /search?q=phone&category=tech&limit=5
```

6. Optional and required query parameters

```
from typing import Optional

@app.get("/items")
def get_items(category: Optional[str] = None):      # optional
    return {"category": category}

@app.get("/search")
def search(q: str):                                # required (no default value)
    return {"query": q}
```

7. Combining path and query parameters

```
@app.get("/users/{user_id}")
def get_user(user_id: int, details: bool = False):
    return {"user_id": user_id, "details": details}
# Example URL: /users/101?details=true
```

Key takeaways

- ✓ Path parameters are part of the URL and required; query parameters are optional and passed after ?.
- ✓ You can use multiple of each, set default values, and combine them together.
- ✓ FastAPI automatically handles type validation. Useful for building flexible and powerful APIs.

Request Body + Pydantic Models

"Define. Validate. Parse. Real-world ready."

1. What is a request body?

- Data sent by the client in the HTTP request (usually in POST, PUT, PATCH).
- Used to send complex data like JSON, instead of just URL parameters.
- In FastAPI we use Pydantic models to define and validate the request body.

2. What is Pydantic?

Pydantic = Data Validation + Type Hints + Automatic Parsing. It lets you define models with type hints, automatically validates and converts incoming data, and if the data is invalid, FastAPI returns a clear error response (422 Unprocessable Entity).

3. Simple request body example

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class User(BaseModel):
    name: str
    age: int
    email: str

@app.post("/users")
def create_user(user: User):
    return {"message": "User created", "user": user}
```

4. Example request (JSON)

```
POST /users
Content-Type: application/json

{ "name": "Nikhil", "age": 24, "email": "nikhil@example.com" }

# Response (success)
{ "message": "User created",
  "user": { "name": "Nikhil", "age": 24, "email": "nikhil@example.com" } }
```

5. Pydantic model with validation

```
from pydantic import BaseModel, EmailStr, Field

class User(BaseModel):
    name: str = Field(..., min_length=3, max_length=50)
    age: int = Field(..., gt=0, lt=120)
    email: EmailStr # validates email format
    is_active: bool = True
```

`Field()` adds extra validation rules, `EmailStr` ensures a valid email, `gt` = greater than, `lt` = less than. You can also set default values.

6. Nested models

```
class Address(BaseModel):
    city: str
    pincode: int

class User(BaseModel):
    name: str
    age: int
    address: Address # nested model
```

7. List of items in request body

```
from typing import List

class Item(BaseModel):
    name: str
    price: float

@app.post("/items")
def create_items(items: List[Item]):
    return {"message": "Items received", "items": items}
```

Key takeaways

- ✓ Use Pydantic models to define the request body; Pydantic validates and parses data.
- ✓ Add custom validation with `Field()`. Supports nested models and lists.
- ✓ Returns clear error messages for invalid data, making your API robust.

Response Models + Status Codes

"Clear responses make better APIs."

1. What are response models?

- They define the structure of the data returned by the API.
- They help in data validation, serialization and automatic documentation (Swagger UI).
- Even if your function returns extra fields, FastAPI filters the response based on the model.
- We use Pydantic models for response models.

2. Basic response model example

```
class UserResponse(BaseModel):
    id: int
    name: str
    email: str

@app.get("/users/{user_id}", response_model=UserResponse)
def get_user(user_id: int):
    return {
        "id": user_id,
        "name": "Nikhil",
        "email": "nikhil@example.com",
        "extra_field": "this will be ignored"    # not in model
    }
```

3. Response model with a list

```
from typing import List

@app.get("/users", response_model=List[UserResponse])
def get_users():
    return [
        {"id": 1, "name": "Nikhil", "email": "nikhil@example.com"},
        {"id": 2, "name": "Payal", "email": "payal@example.com"}
    ]
```

4. Using the status_code parameter

```
from fastapi import status

@app.post("/users", response_model=UserResponse, status_code=status.HTTP_201_CREATED)
def create_user():
    return {"id": 1, "name": "Nikhil", "email": "nikhil@example.com"}
```

5. Common HTTP status codes

Code	Constant (fastapi.status)	Meaning
200	HTTP_200_OK	Successful GET request
201	HTTP_201_CREATED	Resource created successfully
204	HTTP_204_NO_CONTENT	Successful request, no content
400	HTTP_400_BAD_REQUEST	Invalid request
401	HTTP_401_UNAUTHORIZED	Authentication required
403	HTTP_403_FORBIDDEN	Access denied
404	HTTP_404_NOT_FOUND	Resource not found
422	HTTP_422_UNPROCESSABLE_ENTITY	Validation error
500	HTTP_500_INTERNAL_SERVER_ERROR	Server error

6. Custom response with JSONResponse

```
from fastapi.responses import JSONResponse

@app.get("/error")
def custom_error():
    return JSONResponse(
        status_code=404,
        content={"error": "User not found",
                 "message": "The requested user does not exist"}
    )
```

Use JSONResponse when you want to return a custom response format.

7. Key takeaways

- ✓ Use response_model to define the response structure; FastAPI filters extra fields not present in the model.
- ✓ Use status_code to set the appropriate HTTP status code; use Pydantic for both request and response.
- ✓ Common codes: 200, 201, 400, 401, 403, 404, 422, 500. Better responses make your API consistent and professional.

Pydantic Validation + Schemas

"Pydantic helps you validate, parse and manage data using Python type hints."

1. Why Pydantic for validation?

- Automatically validates incoming data using Python type hints (int, str, etc.).
- Provides clear error messages, converts and parses data to correct types.
- Helps in request and response models, and is used by FastAPI internally. Less bugs, more reliable APIs.

2. Basic Pydantic model (schema) and 3. using it in FastAPI

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class User(BaseModel):          # a schema: defines structure and data types
    name: str
    age: int
    email: str

@app.post("/users")
def create_user(user: User):    # FastAPI validates the request body with this schema
    return {"message": "User created", "user": user}
```

4. Validation in action

```
# Invalid request (age is a string)          # Error response (422)
POST /users
{ "name": "Nikhil",
  "age": "twenty",
  "email": "nikhil@example.com" }
{
  "detail": [{
    "loc": ["body", "age"],
    "msg": "Input should be a valid integer",
    "type": "int_parsing" }]
}
```

FastAPI + Pydantic automatically return a **422** error for invalid data.

5. Advanced validation examples

```
from pydantic import BaseModel, EmailStr, Field, conint

class User(BaseModel):
    name: str = Field(..., min_length=3, max_length=50)
    age: conint(ge=0, le=120)      # between 0 and 120
    email: EmailStr                # must be a valid email
    is_active: bool = True
    bio: str | None = Field(None, max_length=200)
```

`Field()` adds extra validation rules; `EmailStr` ensures a valid email; `conint()` adds numeric constraints. You can also use `constr()`, `conlist()`, `confloat()`, etc.

6. Schema for response (separate model)

```
class UserResponse(BaseModel):
    id: int
    name: str
    age: int
    email: EmailStr
    is_active: bool
```

Create different schemas for request and response. This helps in hiding sensitive fields and returning only required data.

7. Nested schemas

```
class Address(BaseModel):
    city: str
    pincode: int

class User(BaseModel):
    name: str
    age: int
    address: Address      # nested model
```

8. Key takeaways

- ✓ Use Pydantic models (schemas) for data; FastAPI automatically validates request validation.
- ✓ Use `Field()` for custom validation (min, max, etc.) and built-in types like `EmailStr`, `conint()`, `constr()`.
- ✓ Create separate schemas for request and response; supports nested models.
- ✓ Returns clear error messages, making your API more robust, secure and maintainable.

Headers, Cookies + Form Data

"FastAPI gives easy access to headers, cookies and form data using simple dependencies."

1. Request headers

Headers contain metadata about the request. Use `Header()` from `fastapi` to access them. You can define defaults, make them optional or convert to a specific type.

```
from fastapi import FastAPI, Header
app = FastAPI()

@app.get("/headers")
def get_headers(user_agent: str = Header(None), x_token: str = Header(None)):
    return {"user_agent": user_agent, "x_token": x_token}
# Example request headers: User-Agent: Chrome X-Token: abc123
```

2. Optional and required headers

```
@app.get("/items")
def read_items(
    x_token: str = Header(...),      # required (use ... to make it required)
    user_agent: str = Header(None)   # optional (use None to make it optional)
):
    return {"token": x_token, "user_agent": user_agent}
```

3. Cookies

Cookies are key-value pairs stored in the user's browser. Use `Cookie()` to read cookies from the request and `Response.set_cookie()` to set them.

```
from fastapi import Cookie, Response

@app.get("/cookies")
def read_cookies(session_id: str = Cookie(None)):
    return {"session_id": session_id}

@app.get("/set-cookie")
def set_cookie(response: Response):
    response.set_cookie(key="session_id", value="abc123", httponly=True, max_age=3600)
    return {"message": "Cookie set successfully"}
```

4. Form data

Used when submitting data from HTML forms (`application/x-www-form-urlencoded` or `multipart/form-data`). Use `Form()` to receive it. Commonly used for login forms, file uploads, etc.

```
from fastapi import FastAPI, Form

@app.post("/login")
def login(username: str = Form(...), password: str = Form(...)):
    return {"message": "Login successful", "username": username}
```

5. File upload with form data (multipart)

Use `UploadFile` and `File()` along with `Form()` for other fields. Content-Type: `multipart/form-data`.

```
from fastapi import FastAPI, File, UploadFile, Form

@app.post("/upload")
async def upload_file(name: str = Form(...), file: UploadFile = File(...)):
    return {"filename": file.filename, "name": name,
            "message": "File uploaded successfully"}
```

6. Key takeaways

- ✓ Use `Header()` to access request headers; use `Cookie()` to read cookies and `Response.set_cookie()` to set cookies.
- ✓ Use `Form()` to handle form data; use `File()` and `UploadFile()` for file uploads.
- ✓ You can make headers, cookies and form fields optional or required.
- ✓ These features help you build real world, production ready APIs.

Authentication + OAuth2 + JWT

"Authentication ensures who you are. Authorization decides what you can do."

1. Why authentication?

Protects your API from unauthorized access and ensures only valid users can reach protected routes. **JWT** (JSON Web Token) is a common way to handle authentication; **OAuth2** is a standard for authentication and authorization; FastAPI supports it via `OAuth2PasswordBearer`.

2. Install packages, 3. user model and fake DB, 4. password hashing

```
pip install fastapi[all] python-jose[cryptography] passlib[bcrypt] # jose = JWT, passlib = hashing

from pydantic import BaseModel
from typing import Optional
from passlib.context import CryptContext

class User(BaseModel):
    username: str
    disabled: Optional[bool] = None

fake_users_db = {"nikhil": {"username": "nikhil", "hashed_password": "$2b$12$examplehashedpassword"}}

pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")
def verify_password(plain, hashed): return pwd_context.verify(plain, hashed)
def get_password_hash(password): return pwd_context.hash(password)
```

Always hash passwords. Never store plain text passwords. In real projects use a proper database instead of a fake DB.

5. Create JWT token

```
from jose import JWTError, jwt
from datetime import datetime, timedelta

SECRET_KEY = "your-secret-key" # use a strong key in production
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 30

def create_access_token(data: dict):
    to_encode = data.copy()
    expire = datetime.utcnow() + timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
    to_encode.update({"exp": expire})
    return jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)
```

6. OAuth2 scheme and 7. login endpoint (get token)

```
from fastapi import FastAPI, Depends, HTTPException, status
from fastapi.security import OAuth2PasswordBearer, OAuth2PasswordRequestForm

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token") # tokenUrl = endpoint that gives the JWT
app = FastAPI()

@app.post("/token")
async def login(form_data: OAuth2PasswordRequestForm = Depends()):
    user = fake_users_db.get(form_data.username)
    if not user or not verify_password(form_data.password, user["hashed_password"]):
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED,
                            detail="Incorrect username or password")
    access_token = create_access_token(data={"sub": user["username"]})
    return {"access_token": access_token, "token_type": "bearer"}
```

8. Protecting routes

```
async def get_current_user(token: str = Depends(oauth2_scheme)):
    credentials_exception = HTTPException(status_code=status.HTTP_401_UNAUTHORIZED,
                                          detail="Could not validate credentials")

    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        username: str = payload.get("sub")
        if username is None: raise credentials_exception
        user = fake_users_db.get(username)
        if user is None: raise credentials_exception
        return User(username=username)
    except JWTError:
        raise credentials_exception

@app.get("/users/me") # protected: requires a valid token
async def read_users_me(current_user: User = Depends(get_current_user)):
    return {"message": f"Hello {current_user.username}"}
```

9. How it works (flow)

1. User sends username + password to /token	2. Server validates credentials	3. Server returns JWT token	4. User sends token in Authorization header for protected routes
---	---------------------------------	-----------------------------	--

10. Key takeaways

- ✓ Use `OAuth2PasswordBearer` and `JWT` for stateless authentication; always hash passwords with `passlib`.
- ✓ Set a strong secret key and short token expiry. In production use a real database and proper user management.

Middleware + CORS

"Middleware runs logic before and after each request; CORS lets your API accept requests from other origins (like your frontend)."

1. What is middleware?

- Code that runs before and after the request reaches your route.
- Used for logging, authentication, timing, modifying request/response, etc.
- In FastAPI you can create custom middleware or use built-in middleware classes.

2. Simple middleware example

```
from fastapi import FastAPI, Request
app = FastAPI()

@app.middleware("http")
async def log_requests(request: Request, call_next):
    print(f"Request: {request.method} {request.url}")
    response = await call_next(request)
    print(f"Response Status: {response.status_code}")
    return response          # logs every request and response

@app.get("/")
def home():
    return {"message": "Hello from FastAPI"}
```

3. Class based middleware

```
import time
from starlette.middleware.base import BaseHTTPMiddleware

class TimingMiddleware(BaseHTTPMiddleware):
    async def dispatch(self, request: Request, call_next):
        start_time = time.time()
        response = await call_next(request)
        process_time = time.time() - start_time
        print(f"Process Time: {process_time:.4f} seconds")
        response.headers["X-Process-Time"] = str(process_time)
        return response

app.add_middleware(TimingMiddleware)
```

4. What is CORS?

CORS (Cross Origin Resource Sharing) allows your API to accept requests from different domains (origins). Browsers block requests from a different origin by default for security reasons. In FastAPI we use `CORSMiddleware`.

5. Enable CORS in FastAPI

```
from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:5173", "https://myapp.com"], # allows your frontend (e.g. React)
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

6. CORS parameters explained

Parameter	Description
<code>allow_origins</code>	List of allowed origins (domains). Use <code>["*"]</code> to allow all (not recommended in production).
<code>allow_credentials</code>	Allows cookies, authorization headers, etc.
<code>allow_methods</code>	Allowed HTTP methods (GET, POST, etc.).
<code>allow_headers</code>	Allowed headers (e.g. Content-Type, Authorization).

7. Real world example (React frontend)

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:3000"], # React dev server
    allow_credentials=True,
    allow_methods=["GET", "POST", "PUT", "DELETE"],
    allow_headers=["*"],
) # now your React app running on port 3000 can call this API
```

8. Key takeaways

- ✓ Middleware runs before and after each request; use it for logging, authentication, timing, etc.
- ✓ CORS allows cross origin requests from frontend applications; use `CORSMiddleware` to configure it.
- ✓ Be careful while allowing all origins (`["*"]`) in production.

Exception Handling + Custom Errors

"Good error handling does not just show errors, it gives meaningful responses and improves user experience."

1. Why exception handling?

- Prevents your application from crashing and gives meaningful error messages to the client.
- Helps in debugging and logging, and improves API reliability and user experience.
- In FastAPI we can handle built-in exceptions or create custom exceptions.

2. Built-in HTTPException

```
from fastapi import FastAPI, HTTPException
app = FastAPI()

@app.get("/items/{item_id}")
def get_item(item_id: int):
    if item_id <= 0:
        raise HTTPException(status_code=400, detail="Item ID must be greater than 0")
    return {"item_id": item_id, "name": "Sample Item"}
```

3. Common HTTP status codes

Code	Meaning
400	Bad Request (invalid input)
401	Unauthorized (authentication required)
403	Forbidden (you don't have permission)
404	Not Found (resource not found)
409	Conflict (resource already exists)
422	Unprocessable Entity (validation error)
500	Internal Server Error (unexpected error)

4. Custom exception class and 5. exception handler

```
from fastapi import Request
from fastapi.responses import JSONResponse

class ItemNotFoundError(Exception):
    def __init__(self, item_id: int):
        self.item_id = item_id
        self.message = f"Item with id {item_id} not found"

@app.exception_handler(ItemNotFoundError) # defines how the custom exception is handled
async def item_not_found_handler(request: Request, exc: ItemNotFoundError):
    return JSONResponse(status_code=404,
                        content={"error": "ItemNotFound", "message": exc.message, "item_id": exc.item_id})

@app.get("/products/{item_id}")
def get_product(item_id: int):
    if item_id != 1:
        raise ItemNotFoundError(item_id)
    return {"item_id": item_id, "name": "Product 1"}
```

6. Handling validation errors (request validation)

```
from fastapi.exceptions import RequestValidationError

@app.exception_handler(RequestValidationError)
async def validation_exception_handler(request: Request, exc: RequestValidationError):
    return JSONResponse(status_code=422,
                        content={"error": "ValidationError", "message": "Invalid input data",
                                "details": exc.errors()})
```

7. Global exception handler (catch all)

```
@app.exception_handler(Exception)
async def global_exception_handler(request: Request, exc: Exception):
    return JSONResponse(status_code=500,
                        content={"error": "InternalServerError",
                                "message": "Something went wrong. Please try again later."})
```

8. Key takeaways

- ✓ Use HTTPException for simple error responses; create custom exception classes for application specific errors.
- ✓ Use @app.exception_handler to handle custom and global exceptions; handle validation errors using RequestValidationError.
- ✓ Return meaningful error messages with proper status codes.

Async Programming + Background Tasks

"Async makes your API handle more requests efficiently, and Background Tasks let you run non-critical work without blocking the response."

1. Why async programming?

- Allows non-blocking I/O operations: while waiting (database query, API call), other requests can be processed.
- Helps handle more concurrent requests. In FastAPI use `async` and `await` with path operations.
- Ideal for I/O bound tasks (database calls, external API requests, file operations).

2. Sync vs async example

```
import time, asyncio
from fastapi import FastAPI
app = FastAPI()

@app.get("/sync")                                # Synchronous (blocking)
def sync_endpoint():
    time.sleep(3)                                # blocks the thread; server cannot handle other requests
    return {"message": "This is a sync endpoint"}

@app.get("/async")                              # Asynchronous (non-blocking)
async def async_endpoint():
    await asyncio.sleep(3)                       # other requests can be processed while waiting
    return {"message": "This is an async endpoint"}
```

3. When to use async?

- Use for I/O bound operations (database, external APIs, file read/write, network calls). Not useful for CPU bound tasks (use background tasks or separate workers).
- Use async with libraries that support it: **httpx** (async HTTP client), **databases / asyncpg** (async DB queries), **motor** (MongoDB), **aiofiles** (async file operations).
- If a library does not support async, using async will not give any benefit.

4. Background tasks in FastAPI

- Run small tasks in the background after sending the response (emails, logging, file processing).
- Implemented using `BackgroundTasks` from FastAPI; they run in the same process (not for heavy or long-running tasks).

5. Simple background task example

```
from fastapi import FastAPI, BackgroundTasks
import time

def write_log(message: str):
    with open("log.txt", "a") as f:
        f.write(f"{message}\n")
        time.sleep(2)                                # simulate a time-consuming task

@app.post("/send-notification")
async def send_notification(background_tasks: BackgroundTasks):
    background_tasks.add_task(write_log, "User signed up")
    return {"message": "Notification will be processed in background"} # sent immediately
```

6. Use case: send email in background

```
def send_email(to_email: str, subject: str, body: str):
    # integrate with your email service here
    print(f"Sending email to {to_email}")

@app.post("/signup")
async def signup(email: str, background_tasks: BackgroundTasks):
    background_tasks.add_task(send_email, email, "Welcome", "Thanks for signing up!")
    return {"message": "User created. Welcome email will be sent."}
```

7. Limitations

- Not suitable for long-running or heavy CPU tasks; for heavy tasks use a task queue like **Celery** with Redis/RabbitMQ.
- Tasks run in the same process, so if the app restarts, pending tasks may be lost. Use for small, quick, non-critical jobs only.

8. Key takeaways

- ✓ Use `async` and `await` for non-blocking I/O; always use async compatible libraries for real performance gains.
- ✓ Use `BackgroundTasks` for small, non-critical tasks; for heavy or long-running tasks use Celery or a task queue.

Quick Revision Cheat Sheet

"Small steps. Big possibilities."

Setup and run

```
pip install fastapi "uvicorn[standard]"
python -m venv .venv && source .venv/bin/activate # Windows: .venv\Scripts\activate
uvicorn app.main:app --reload # docs: /docs (Swagger) and /redoc
```

HTTP methods and decorators

Decorator	Method	Purpose
@app.get()	GET	Read data (idempotent)
@app.post()	POST	Create data
@app.put()	PUT	Replace a resource (idempotent)
@app.patch()	PATCH	Partial update
@app.delete()	DELETE	Remove a resource (idempotent)

Where does the data come from?

Source	How to declare it	Example
Path parameter	{name} in the path + typed argument	/users/{user_id}
Query parameter	Argument with a default value	/items?skip=0&limit=10
Request body	Pydantic model argument	user: User
Header	Header()	x_token: str = Header(...)
Cookie	Cookie()	session_id: str = Cookie(None)
Form field	Form()	username: str = Form(...)
File upload	UploadFile + File()	file: UploadFile = File(...)

Status codes to remember

200 OK	201 Created	204 No Content	400 Bad Request	401 Unauthorized	403 Forbidden	404 Not Found	422 Validation	500 Server Error
--------	-------------	----------------	-----------------	------------------	---------------	---------------	----------------	------------------

Pydantic validation toolkit

BaseModel • Field(..., min_length, max_length, gt, lt) • EmailStr • conint(gt, le) • nested models • List[Model] • separate request and response schemas.

Security, middleware and errors

- **Auth:** OAuth2PasswordBearer + JWT; hash passwords with passlib; strong secret key; short expiry.
- **Middleware:** @app.middleware("http") or BaseHTTPMiddleware for logging and timing.
- **CORS:** CORSMiddleware with allow_origins, allow_credentials, allow_methods, allow_headers; avoid "*" in production.
- **Errors:** HTTPException, custom exception classes, @app.exception_handler, RequestValidationError, global Exception handler.

Async and background work

- Use async def + await for I/O bound work with async libraries (httpx, asyncpg, motor, aiofiles).
- BackgroundTasks for small jobs after the response; Celery + Redis/RabbitMQ for heavy or long-running jobs.

Recommended learning path

- ✓ Build a small CRUD API with Pydantic models and proper status codes.
- ✓ Add a database, routers/services structure and environment-based config.
- ✓ Secure it with JWT, add CORS for your frontend, and handle errors globally.

Thanks for reading! Follow on Instagram: @____raviiiiiii for more learning content. **Learn. Build. Repeat.**